

Telecom Network Intelligence System

End-to-End Capstone Project
Design Document

Engineering Team:

Arya Putatunda

Bhavmeet Singh Ahuja

Karan Sood

PRODAPT

1. Executive Summary

The Telecom Network Intelligence System is a unified, end-to-end data platform built to provide actionable network intelligence for telecom operations. Processing over 15 million records, the system enables operational leaders to monitor network congestion, view daily/hourly usage trends, and proactively identify cell-tower zones at risk of overload using Machine Learning.

2. Architecture & Data Flow

The system is built on a modern, distributed data stack orchestrated by Apache Airflow across hybrid execution environments:

- **Landing Zone:** Ingests raw CSV data into `data/landing/`.
- **Raw Zone:** Files validated for schema integrity are moved to `data/raw/`.
- **Processing (PySpark):** A distributed PySpark job processes raw records, enriches them with spatial metadata via Broadcast Joins, and outputs to the `data/processed/` zone.
- **Data Warehouse:** The transformed data is loaded into a MySQL/PostgreSQL Star Schema.
- **Serving & ML:** A FastAPI backend exposes the warehouse data and serves a Random Forest ML model predicting network congestion. The frontend is built in React.

3. Storage Strategy Document

Our storage choices at each layer are specifically tailored for performance and scale:

- **Raw Layer (Why CSV?):** CSVs are the standard output format for legacy telecom switches and cell towers. They are human-readable, universally supported for raw ingestion, and allow for quick initial inspection before parsing.
- **Processed Layer (Why Parquet? Why partition by date?):** Parquet is a columnar storage format highly optimized for Big Data analytics. It vastly reduces storage footprints via compression and speeds up read times for PySpark. Partitioning by date allows downstream queries to skip irrelevant data (Predicate Pushdown), significantly boosting performance when analyzing specific days or weeks.
- **Warehouse Layer (Why a Star Schema?):** We use a Star Schema rather than a single flat table to avoid data redundancy and maintain referential integrity. The high-volume network events are stored in the central Fact table, while descriptive metadata (like region names and time

hierarchies) are isolated in Dimension tables. This optimizes aggregation queries for the analytical dashboard.

4. Batch vs Streaming Decision

Telecom Data Flow	Classification	Justification
Network activity logs from cell towers	Streaming	Cell towers generate constant, real-time events. Processing this as a stream allows for immediate detection of outages or sudden overloads.
Daily usage summary	Batch	Summaries are intrinsically bound to a fixed 24-hour window. Running a single scheduled batch job at midnight is computationally efficient and perfectly suited for daily reporting.
Monthly billing reports	Batch	Billing is a scheduled, high-volume process that occurs on a fixed cycle (end of the month). Immediate processing is unnecessary.
Network congestion alerts	Streaming	Congestion alerts require real-time or near-real-time latency. If a tower drops or overloads, operations must be notified instantly, not hours later.
Customer dashboard data	Batch / Micro-Batch	Customer dashboards typically display historical trends and summarized usage patterns, which can be effectively refreshed at scheduled intervals (e.g., hourly micro-batches).

5. Warehouse Modelling (Star Schema)

The analytical data is structured to optimize the dashboard's sub-second querying requirements.

Fact Table: **fact_usage**

- **usage_id** (BIGINT, Primary Key)
- **time_id** (BIGINT, Foreign Key)
- **grid_id** (BIGINT, Foreign Key)
- **call_count** (DOUBLE)
- **sms_count** (DOUBLE)
- **internet_usage** (DOUBLE)

Dimension Tables

- **dim_time:** time_id (PK), timestamp, date, hour, day_of_week, is_weekend
- **dim_region:** grid_id (PK), region_name, city

6. Machine Learning: Problem Statement

Objective: To predict the likelihood of network congestion (High, Medium, or Low risk) at specific cell-tower grids.

Value to Telecom Operations: By proactively identifying regions at risk of network overload, operations teams can dynamically allocate bandwidth, reroute traffic, or schedule preventative maintenance before users experience dropped calls or degraded internet speeds. This drastically improves customer satisfaction and network reliability.

Labeling Rule: Since the dataset is unlabeled, we apply a statistical threshold based on historical internet and call volume distributions for each region. Usage exceeding the 90th percentile of the region's historical average is labeled "High Risk", between the 75th and 90th percentile as "Medium Risk", and below the 75th percentile as "Low Risk".

7. Failure Handling Scenarios

The Airflow orchestration layer is engineered to handle disruptions gracefully. *(Note: Screenshots verifying these scenarios are provided in the team's presentation and repository documentation.)*

1. **Missing Input File:** The `detect_files` task fails safely, halting the pipeline to prevent downstream processes from executing on empty sets. The Airflow UI marks the task red and issues an alert.
2. **Corrupt CSV Rows:** The `validate_files` task detects schema mismatches or nulls in required fields, moving the corrupt file to the `data/rejected/` zone without crashing the entire batch process.
3. **Spark Job Crash:** If PySpark runs out of memory or crashes, Airflow automatically retries the task up to 3 times before failing the DAG run, triggering a failure notification payload.